



DOM shared between the webpage scripts and the content scripts

according to user actions like clicks or key presses and browser actions like loading a page, closing a page etc.

When we use extensions over some webpage, the entities of the page may be changed in which they are presented before the user. Content scripts are basically javascripts that run operate upon the content in the web page in current context. But, the page might have its own javascripts that perform function that the site provides. How does it work out then?

The DOM (Document Object Model) is shared between the two scripts and both the scripts can operate upon the DOM independently.

With this ability given to the content scripts, they can be used to perform tasks like resizing text, changing element attributes, linking textual hyperlinks that are unlinked, downloading all media files that are present in the page and anything at all that can be done with the DOM. Changes made by any of the two scripts viz. the web page scripts and the content scripts to the DOM are visible to each of them.

Important point to be noted here is that the scripts should remain isolated from each other. They should not interfere with each other's functionality. The scripts may also use different versions of a given library, care is taken that they do not interfere with each other in any way. Isolation provided also takes care that the webpage scripts do not get access to the privileged API that the extensions are capable of using. Hence the data accessible through that API and sensitive user credentials are prevented from being leaked. All the content scripts are isolated from the webpage scripts and also from the other content scripts for different extensions.

Injecting your content scripts

The content scripts that needed to be injected as routine into the code should be registered in the manifest.